

# Capital Bike Demand Audit

## Sample task overview

*Long-horizon Harbor coding environment · CPU ML engineering*

This document covers what the task is, what the agent has to do, how it is graded, how frontier models performed and what ships in the folder. The task files and full rollout traces are included alongside it.

### Task identification

| Field         | Overview                                                                                                              |
|---------------|-----------------------------------------------------------------------------------------------------------------------|
| Task ID       | capital-bike-demand-audit                                                                                             |
| Category      | CPU ML engineering: time-series demand forecasting and production release operations                                  |
| Scenario      | A frozen September–December 2012 hourly bike-demand forecast release that has to be repaired, recovered and shipped   |
| Task format   | Editable workspace, instruction, visible tests, operations CLI, protected service, verifier and reference solutions   |
| Real data     | UCI Bike Sharing hourly table (17,379 records, CC BY 4.0)                                                             |
| Compute       | 2 vCPU, 7 GB RAM, no GPU; network available to the agent (harness requirement); not used by the task; grading offline |
| Agent budget  | 9,000 seconds; verifier 2,400 seconds                                                                                 |
| Agent harness | mini-swe-agent, used for both models' rollouts                                                                        |
| Deliverable   | A complete signed release trajectory ending in submit, and the environment-owned terminal forecast it produces        |

### Description

The agent inherits a live forecasting incident. A bike-share operator's four-month hourly demand forecast is frozen: the inherited pipeline cannot reconcile four incompatible history exports, relies on a weak retrospective model, and publishes 80% prediction intervals that do not hold up when replayed forward. Local checks look reasonable, but the production horizon is still wrong.

To solve it the agent has to do the **full engineer's job**: investigate the exports and evidence through a protected operations service, commit a hypothesis backed by signed evidence, repair the pipeline across multiple files, deploy an immutable snapshot, recover from a deliberately injected stale-cache fault, redeploy, pass audits, and drive the release through canary and production to submission. Every action is signed by a root-owned service and replayed by the verifier, so a code drop or a forged log earns nothing.

The core technical difficulty is **forecasting across a four-month horizon from a limited calendar range**. A standard tree model trained on the visible history looks fine on the public backtest and passes the operational validations, but it does not carry demand levels forward into late 2012. A passing solution has to work out a sound horizon treatment (a year bridge), represent working-day commute hours properly, and calibrate its intervals from forward residuals, without ever seeing terminal labels.

## The incident the agent inherits

- **Four incompatible exports.** The labelled history is split across files that differ in delimiter (comma, semicolon, pipe), column names, weather units and duplicate behaviour. A schema registry describes how to normalise them into 11,539 unique rows.
- **Outcome leakage risk.** cnt, casual and registered are all outcomes. They exist in history but are absent at issue time and must never be used as predictors.
- **A weak model and uncalibrated intervals.** The starter model does not bridge the calendar gap to the production horizon, and its intervals are not calibrated from forward residuals.
- **A staged operational fault.** The first protected validation is deliberately dirty because the served cache is stale. The agent must read signed diagnostics and apply the matching recovery family.

## What the agent has to do

The work is a causally ordered, signed walk through the operations service. The direct path is **34 actions**; a candidate that is viable but still off after recovery gets one bounded revision cycle, for **43 actions**.

| Phase                   | What happens                                                                                                                                                                                                                                        | Operations                                                    |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| 1. Investigation        | Inspect the contract and exports, profile the current candidate, run five protected probes (schema, leakage, year bridge, commute, horizon) and commit a hypothesis citing all five evidence receipts. Editing code before this locks the incident. | inspect.*, profile.current, probe.* ×5, hypothesis.commit     |
| 2. Repair and rehearsal | Change at least two implementation files, pass visible tests and the public backtest, attest the candidate and reproduce it in the runtime.                                                                                                         | test.visible, backtest.replay, candidate.attest, reproduce    |
| 3. First deployment     | Open an immutable stage, prepare and commit the deployment, and validate. This validation is forced dirty.                                                                                                                                          | stage.open, deploy.prepare, deploy.commit, validate           |
| 4. Recovery             | Read signed diagnostics, plan and apply the recovery family they require, verify it and audit the cache.                                                                                                                                            | diagnostics.read, recovery.plan / apply / verify, cache.audit |
| 5. Second deployment    | Reproduce against the advanced runtime and deploy again; validate.                                                                                                                                                                                  | reproduce, stage.open, deploy.*, validate                     |
| 6. Audits and release   | Horizon and interval audits, canary promotion and validation, production promotion, freeze and submit.                                                                                                                                              | audit.*, promote.*, validate.canary, freeze, submit           |

Validation is rate-limited to three accepted calls and returns only clean or dirty plus an operational signal, never labels or row-level errors. Out-of-order operations, a wrong recovery family or a mismatched receipt are rejected without advancing, so the agent can correct and retry but cannot skip steps.

## Example prompt

Opening of the task instruction given to the agent:

*I am handing you a live hourly bike-demand forecast incident. The September through December 2012 release is frozen because the inherited pipeline cannot reconcile its mixed history exports, uses a weak retrospective model, and publishes intervals that do not survive a forward replay. Ordinary local checks can look reasonable while the four-month production horizon is still wrong. ... The operations service is authoritative. It owns the runtime cache, protected validation waves, staged snapshots, signed action history, and final replay. Editing code alone is not a delivery.*

The full instruction (task/instruction.md) specifies the ordered walk, the evidence commitment format, the recovery rules and the terminal bars below.

## Time, QA and difficulty calibration

Difficulty was measured, not asserted: each model was run four times (pass@4) on the mini-swe-agent harness inside the real task environment, and every attempt was graded by the same verifier.

| Model                            | Strict passes | Mean reward | Note                               |
|----------------------------------|---------------|-------------|------------------------------------|
| Claude Fable 5 (xhigh reasoning) | 4/4           | 1.00        | 62–69 steps per run, 21–26 minutes |
| Grok 4.7 (xhigh reasoning)       | 1/4           | 0.25        | 34–48 steps per run                |

**Target profile: a clear frontier lead.** The task is calibrated so that Grok 4.7 lands in a 25–50% band (1–2 of 4 attempts), solvable but discriminating, while the frontier Claude model solves it reliably. Here Claude Fable 5 solved 4 of 4 against Grok 4.7's 1 of 4, a 75-percentage-point lead in strict pass rate.

**Terminal bars.** At submit the service replays the frozen production snapshot over all 2,888 terminal requests (September–December 2012). To pass, the forecast must meet every bar:

| Metric                       | Bar           |
|------------------------------|---------------|
| Overall RMSE                 | ≤ 65          |
| Worst calendar-month RMSE    | ≤ 72          |
| Working-day commute-hour MAE | ≤ 76          |
| 80% interval coverage        | 70% to 98%    |
| Mean interval width          | ≤ 250 rentals |

**Per-attempt terminal results** (values outside a bar in bold):

| Attempt            | Result | Overall RMSE | Worst month | Commute MAE | Coverage | Mean width |
|--------------------|--------|--------------|-------------|-------------|----------|------------|
| Fable 5 · SjAdFe   | Pass   | 57.00        | 66.67       | 64.71       | 82.4%    | 124.2      |
| Fable 5 · ZReeuEa  | Pass   | 56.35        | 65.64       | 60.56       | 87.0%    | 143.8      |
| Fable 5 · ha6HwxZ  | Pass   | 57.38        | 67.31       | 57.03       | 84.6%    | 125.6      |
| Fable 5 · i8iyRMh  | Pass   | 60.43        | 70.92       | 72.92       | 86.5%    | 144.2      |
| Grok 4.7 · 5CxJPnC | Pass   | 57.24        | 65.95       | 64.15       | 91.6%    | 174.3      |

| Attempt            | Result | Overall RMSE | Worst month  | Commute MAE  | Coverage | Mean width |
|--------------------|--------|--------------|--------------|--------------|----------|------------|
| Grok 4.7 · E6HCixe | Fail   | <b>74.77</b> | <b>79.55</b> | <b>89.18</b> | 85.9%    | 182.3      |
| Grok 4.7 · b77Jf7X | Fail   | <b>70.50</b> | <b>80.93</b> | 74.36        | 80.8%    | 122.1      |
| Grok 4.7 · kYzwjvj | Fail   | <b>66.57</b> | <b>73.83</b> | 75.40        | 82.2%    | 137.1      |

## Why the model failed

Grok 4.7's three failed attempts are the failure mode the task is designed to catch. In each one the agent **completed the whole signed release**: the evidence chain, the forced recovery, both deployments, the audits, canary and production promotion, and submit all verified cleanly (gate, chain and binding checks all ok). The failure is purely in the forecast itself: the frozen snapshot misses the overall and worst-month RMSE bars on the four-month horizon, and one attempt misses the commute bar too.

That is by design. The protected validation waves use broad value bars, so they confirm a viable candidate without revealing the final discriminator. An agent that treats a clean validation as proof of correctness, instead of reasoning about how its model carries demand across the horizon, ships a release that looks healthy and is wrong. The pass rate therefore reflects forecasting judgement under partial observability, not whether the agent can follow a checklist.

## Grading design

Reward is binary (1 only if every component passes), but the verifier reports each component separately, so every attempt yields a dense diagnostic record rather than a single bit.

| Component         | Overview                                                                                                                                                                                            |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Technical result  | The frozen snapshot is replayed by the service over environment-owned history and all 2,888 terminal requests; values are checked against verifier-held labels with tolerance.                      |
| Robustness        | The verifier independently replays the same snapshot under row permutations of history and requests and requires identical results.                                                                 |
| Operational path  | The full signed action chain is replayed from protected, root-owned state; the ordered walk, evidence binding and deployment identities must all verify.                                            |
| Recovery          | The staged stale-cache fault must be diagnosed and cleared with the family named in the signed diagnostics.                                                                                         |
| Multi-file repair | At least two implementation files must change relative to the pristine, hash-bound candidate.                                                                                                       |
| Isolation         | Candidate code runs as an unprivileged user without access to protected requests, labels or service state. Workspace logs, forged receipts, stdout claims and self-written score files are ignored. |

## Failure, retries and limits

### How we determine the agent has failed

Grading is deterministic and strictly binary: reward is 1.0 (pass) or 0.0 (fail). The verifier reads environment-owned state the agent cannot write: the root-owned service record and the terminal the service writes at submit by replaying the **frozen production snapshot**, not the live workspace.

Reward is 1.0 only if every condition holds: the frozen snapshot meets all five terminal bars on the hidden September–December requests, the ordered signed walk was driven end to end, the staged stale-cache fault was diagnosed and cleared with the family the diagnostics required, **and** the release was submitted. Miss any one and the reward is 0.0: a forecast outside a bar, a skipped or reordered step, a wrong recovery, or no submit.

## Retries

- **No free retries of the outcome.** Within an attempt the agent can correct and retry: an out-of-order operation, a wrong recovery family or a mismatched receipt is rejected without advancing, and the agent retries the same operation.
- **Feedback is lossy and rationed.** Validate is limited to three accepted calls and returns only clean or dirty plus an operational signal. A candidate still dirty after recovery gets one bounded revision cycle; a dirty third validation ends the release.
- **submit freezes the terminal.** After submit the service refuses every further operation, so there is no re-submit.
- **Across attempts:** pass@4 means four fully independent attempts, each in a fresh container. They are independent samples used to measure the solve rate, not retries the agent controls. The target is Grok 4.7 in the 25–50% band (1–2 of 4) with the frontier Claude model well ahead; this task measured 4 of 4 for Claude Fable 5 and 1 of 4 for Grok 4.7.

## Step and time limits

- **Time limit:** a hard 9,000 seconds (2 h 30 min) per attempt. An attempt still running at the limit is stopped.
- **Step limit:** none. The mini-swe-agent harness enforces no per-attempt step cap, so an attempt runs until the agent submits or reaches the time limit.
- **Never submitting is a fail, not an error.** An attempt that never submits, or times out first, is graded 0, since there is no frozen terminal to grade. Genuine infrastructure failures, such as a container or harness fault, are reported separately as inconclusive and are never recorded as 0.

## QA process

The task passes the same automated QA pipeline as the rest of the pack. Evidence for this task ships in the rollout folders:

- **Build and gates.** The Docker image builds for each model's evaluation run (digests sha256:80fca149... at 468 MB for Claude Fable 5 and sha256:432353b8... at 1.88 GB for Grok 4.7, recorded in the gates evidence). The Grok image pre-installs the agent harness (uv, mini-swe-agent) after uv setup crashes; no task file differs. The reference solution scores 1 and the untouched starting state scores 0.
- **Three structurally different positive solutions.** Year-bridge histogram gradient boosting, a capped year-bridge random forest and capped year-bridge extra trees each complete the 34-action path and score 1, so grading rewards the outcome rather than the shape of one fix. A fourth positive control starts from the inherited baseline, takes the protected revision cycle, changes two files and also scores 1.
- **Nine negative controls, all required to score 0:** doing nothing; forging a complete trajectory and terminal file in the workspace; changing only one implementation file; a familiar tree model with no year bridge; printing forged success to stdout; choosing a recovery family that does not match the

diagnostics; hanging during inference; tampering with root-owned service state; and trying to read protected requests.

- **Determinism checks.** Repeated runs, malformed outputs, row-order changes, numeric formatting, value tolerance and strict reward bytes are all exercised by the calibration harness.

## Environment and execution

Python CPU container; network available to the agent (harness requirement); not used by the task; grading offline. The agent works in `/workspace/target/`, which holds the labelled history exports and their schema registry, a data card and forecast contract, the starter pipeline (data, features, model, forecast and interval modules), prediction and backtest scripts, visible tests and `env_cli.py`. The operations service runs as root outside the workspace, owns all state, signs every action with a root-only key and runs candidate code as an unprivileged user.

The runtime contract is a single command, `scripts/predict.py`, which must produce deterministic, permutation-invariant output with exactly `instant`, `predicted_cnt`, `lower_80` and `upper_80` for every request.

## Raw base data

| Field            | Overview                                                                                                    |
|------------------|-------------------------------------------------------------------------------------------------------------|
| Source           | UCI Bike Sharing dataset, hourly table (Capital Bikeshare, Washington D.C.)                                 |
| Reference        | <a href="https://archive.ics.uci.edu/dataset/275">archive.ics.uci.edu/dataset/275</a> · DOI 10.24432/C5W894 |
| License          | Creative Commons Attribution 4.0 International                                                              |
| Integrity        | Source SHA-256 verified by the workspace generator; every data surface is reproduced deterministically      |
| Visible history  | 11,539 unique hourly records, January 2011 to April 2012                                                    |
| Protected waves  | May–June 2012 (recovery wave) and July–August 2012 (release wave)                                           |
| Terminal horizon | 2,888 hourly records, September–December 2012                                                               |

## Technical domains / relevant benchmarks

The task combines MLE-bench-style ML engineering (time-series forecasting, leakage control, interval calibration) with SWE-bench and DeepSWE-style multi-file repository repair and Terminal-Bench-style autonomous terminal execution. Its signed, ordered operations, hidden service state and staged fault recovery also exercise T-bench-style stateful tool use, applied to a production ML release rather than a simulated conversation.

## Folder contents

| Folder             | Contents                                                                                                                                                                                                                                                                                                                  |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>task/</code> | The complete Harbor task: <code>instruction.md</code> , <code>task.toml</code> , environment (Dockerfile, protected service, workspace), tests (verifier and hidden labels), solution (reference and two alternate solutions) and <code>_private</code> (workspace generator, calibration harness and negative controls). |

| Folder           | Contents                                                                                                                                                         |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Fable 5 pass@4/  | agg/ holds the aggregate report and every trial's trajectory and verifier output; gold/ holds the build log, image digest and the oracle and no-op gate results. |
| Grok 4.7 pass@4/ | The same agg/ and gold/ layout for the four Grok 4.7 rollouts.                                                                                                   |

## How the task was created

The real-data forecasting problem was chosen first, then the incident was built around it by measurement: the export incompatibilities, the terminal bars and the protected waves were each set so that familiar baselines fail while structurally different correct solutions pass. The task was then packaged with reference and alternate solutions, a bounded-revision positive control, the negative-control battery, protected verification and deterministic build checks.

## Acknowledgement

I confirm that I have reviewed the tasks and believe they are realistic, technically substantive, clearly specified, and appropriately graded. They reflect genuine professional workflows and are not low-effort, artificial, or "slop" tasks.

---

Name: Suleiman Mahmood

Role: Co-Founder

---

Signature: /s/Suleiman Mahmood

Date: 9.28.26

---